ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA

KHOA ĐIỆN - ĐIỆN TỬ



MÔN HỌC

TRÍ TUỆ NHÂN TẠO TRONG ĐIỀU KHIỂN

"THUẬT TOÁN PLA - PERCEPTION LEARNING ALGORITHM"

REPORT HOMEWORK 1

Lớp: L01 Nhóm: 05 HK261

GVHD: TS. PHẠM VIỆT CƯỜNG

STT	Sinh viên	MSSV	Điểm
1	Nguyễn Văn Tuấn Anh	2310131	
2			
3			
4			

TP. Hồ Chí Minh, ngày 23 tháng 09 năm 2026

Mục lục

1 Tóm tắt lý thuyết **2**

1.1 Phân lớp nhị phân . . . 2

1.2 Thuật toán perceptron . . . 2

1.2.1 Cách phân lớp . . . 2

1.2.2 Thuật toán phân lớp Perceptron . . . 3

2 Example 1 **3**

2.1 Block 1: Dataset generation . . . 3

2.2 Block 2: Các hàm của PLA . . . 5

2.2.1 Hàm dấu . . . 6

2.2.2 Dự đoán một điểm . . . 6

2.2.3 Dự đoán toàn dataset . . . 6

2.2.4 Kiểm tra hội tụ . . . 6

2.2.5 Vòng lặp cập nhật . . . 7

2.3 Block 3: Chạy PLA . . . 8

2.4 Block 4: Vẽ đường phân tách . . . 8

2.5 Block 5: Hoạt họa từng lần cập nhật . . . 8

Danh sách hình vẽ

1 Bài toán phân lớp nhị phân trong 2D . . . 2

2 Dataset với N = 5 . . . 4

3 Kết quả 20 điểm bị phân loại là sai trước khi hội tụ. . . . 8

4 Kết quả chạy PLA với $N = 5$ sau 20 lần cập nhật . . . 8

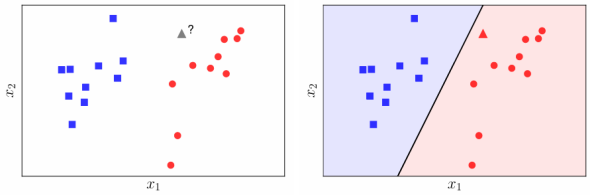
1 Tóm tắt lý thuyết

Chương này trình bày bài toán phân lớp nhị phân và thuật toán học Perceptron (Perceptron Learning Algorithm, PLA).

1.1 Phân lớp nhị phân

Bài toán phân lớp nhị phân là phân loại 2 lớp dữ liệu riêng biệt (hay còn gọi là binary classification).

Giả sử có hai class tương ứng hai tập điểm xanh và đỏ, bài toán đặt ra là từ dữ liệu đã gắn nhãn xanh-đỏ (cho rằng tập 2 dữ liệu này linearly separable), ta cần tìm một bộ phân lớp (classifier) có khả năng dự đoán màu của một điểm dữ liệu mới.



Hình 1: *Bài toán phân lớp nhị phân trong 2D*

Nếu coi mỗi vector đặc trưng là một điểm trong không gian d chiều, bài toán phân lớp coi như bài toán xác định điểm đó thuộc lớp nào. Hay nói cách khác, nếu mỗi lớp chiếm một vùng trong không gian, ta cần xác định ranh giới (boiundary) giữa các vùng này.

1.2 Thuật toán perceptron

1.2.1 Cách phân lớp

- Giả sử $X = [x_1, x_2, \ldots, x_n] \in \mathbb{R}^{d \times N}$ là ma trận vector chứa các điểm huấn luyện.

- x_i là vector đặc trưng thứ i

- Nhãn $y = [y_1, y_2, \ldots, y_n] \in \mathbb{R}^{1 \times N}$ với $y_i = 1$ nếu x_i thuộc lớp 1, $y_i = -1$ nếu x_i thuộc lớp 2

Tại một thời điểm, giả sử có boundary là siêu phẳng có pt:

$$f(\mathbf{x}) = w_1 x_1 + \cdots + w_d x_d + w_0 = \mathbf{w}^T \mathbf{x} + w_0 = 0$$

- Với $\mathbf{w} \in \mathbb{R}^d$ là vector hệ số, w_0 là bias (tự do).

- Dùng **bias trick** để đưa w_0 vào $\mathbf{w}$, ta có siêu phẳng là

$$\tilde{\mathbf{x}} = \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}, \qquad \tilde{\mathbf{w}} = \begin{bmatrix} \mathbf{w} \\ w_0 \end{bmatrix}, \qquad f(\tilde{\mathbf{x}}) = \tilde{\mathbf{w}}^T \tilde{\mathbf{x}} = 0$$

- Nếu w là nghiệm của bài toán perceptron, với data mới x chưa gán nhãn , ta có thể gán nhãn bằng cách:

$$\text{label}(\mathbf{x}) = \begin{cases} 1 & \text{nếu } \mathbf{w}^T \mathbf{x} \geq 0, \\ -1 & \text{o.w.} \end{cases} \tag{1}$$

- Nói cách khác, $\text{label}(\mathbf{x}) = \text{sgn}(\mathbf{w}^T \mathbf{x})$ với sgn là hàm xác định dấu, giả sử rằng $\text{sgn}(0) = 1$.

1.2.2 Thuật toán phân lớp Perceptron

Bằng việc xây dựng hàm mất mát:

$$J(\mathbf{w}) = \sum_{\mathbf{x}_i \in \mathscr{M}} \left(-y_i \mathbf{w}^T \mathbf{x}_i\right)$$

Trong đó $\mathscr{M}$ là tập các điểm dữ liệu bị phân lớp lỗi, tức là các điểm dữ liệu mà label$(\mathbf{x}_i) \neq y_i$.

Và với một điểm dữ liệu x_i bị phân lớp lỗi, hàm mất mát và đạo hàm của nó:

$$J(\mathbf{w}; \mathbf{x}_i, y_i) = -y_i \mathbf{w}^T \mathbf{x}_i, \qquad \nabla_{\mathbf{w}} J(\mathbf{w}; \mathbf{x}_i, y_i) = -y_i \mathbf{x}_i$$

Và tối ưu hàm mất mát bằng Schoolastic Gradient Descent (SGD):

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \left(-y_i \mathbf{x}_i\right) = \mathbf{w} + \eta y_i \mathbf{x}_i \tag{2}$$

2 Example 1

2.1 Block 1: Dataset generation

Listing 1: Tạo dataset 2D

```
# ============================================================
# 1. create 2 classes of 2D points
# ============================================================
np.random.seed(2)

# center values for 2 classes
mean0 = [2, 2]   # +1
mean1 = [4, 2]   # -1

# co-variance
cov = [[0.3, 0.2],
       [0.2, 0.3]]

N = 5  # number of points per class

# create random points for each class following Gaussian distribution
# N rows, 2 columns (each row is a point x, y)
class0 = np.random.multivariate_normal(mean0, cov, N)
class1 = np.random.multivariate_normal(mean1, cov, N)

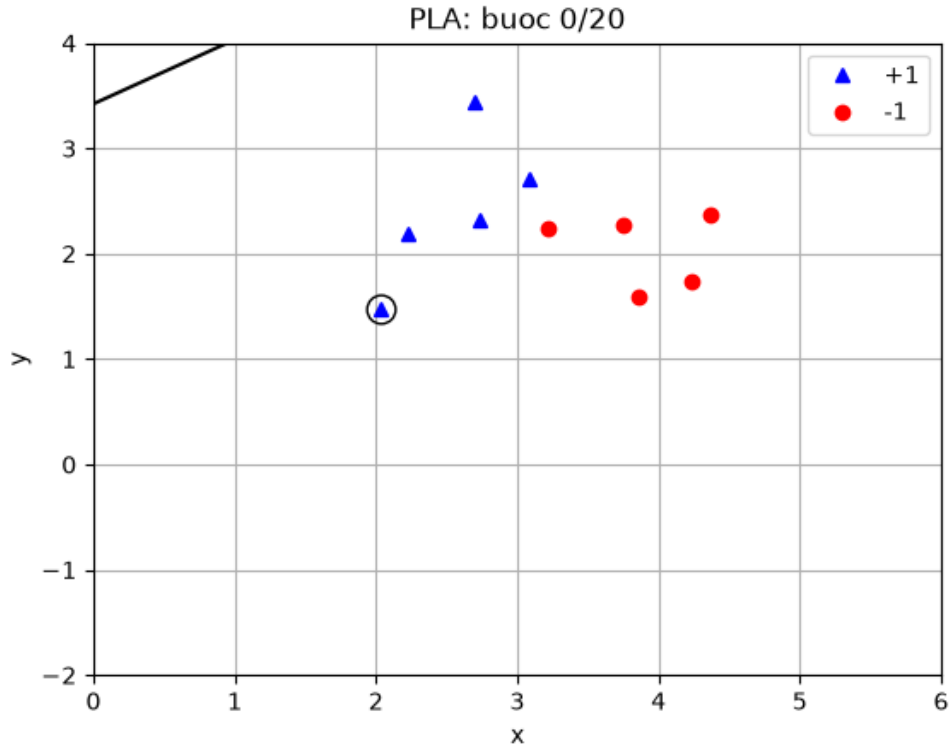
# convert to shape (2 rows, N columns) for easier processing
X0 = class0.T
X1 = class1.T

# combine the two classes into one dataset
# X_points shape = (2, 40)
X_points = np.concatenate((X0, X1), axis=1)

# labels for the two classes: +1 for class0, -1 for class1
# y shape = (1, 40)
```

```
y_pos = np.ones((1, N))
y_neg = -1 * np.ones((1, N))
y = np.concatenate((y_pos, y_neg), axis=1)

# add a row of ones to X_points to create X_bar for bias term
# X shape = (3, 40): row 0 = 1, row 1 = x, row 2 = y
ones_row = np.ones((1, 2 * N))
X = np.concatenate((ones_row, X_points), axis=0)
```



Hình 2: *Dataset với N = 5*

Với $N = 5$, khối này được thực hiện tuần tự như sau. Ký hiệu $\mathbf{x}_i^{(+)}$ là điểm thuộc lớp $+1$ và $\mathbf{x}_i^{(-)}$ là điểm thuộc lớp -1, trong đó $i = 1,\ldots,N$.

1. **Chọn tham số của hai lớp.** Với số chiều $d = 2$, hai vector trung bình và ma trận hiệp phương sai là

$$\mu_+ = \begin{bmatrix} 2 \\ 2 \end{bmatrix}, \qquad \mu_- = \begin{bmatrix} 4 \\ 2 \end{bmatrix}, \qquad \Sigma = \begin{bmatrix} 0.3 & 0.2 \\ 0.2 & 0.3 \end{bmatrix}.$$

   Đặt $N = 5$ và cố định hạt giống ngẫu nhiên bằng np.random.seed(2), nên các mẫu sinh ra được giữ nguyên giữa các lần chạy.

2. **Sinh từng điểm của hai lớp.** Với $i = 1,\ldots,5$, ta lấy

$$\mathbf{x}_i^{(+)} \sim \mathcal{N}(\mu_+, \Sigma), \qquad \mathbf{x}_i^{(-)} \sim \mathcal{N}(\mu_-, \Sigma), \qquad \mathbf{x}_i^{(+)}, \mathbf{x}_i^{(-)} \in \mathbb{R}^2.$$

   Mỗi $\mathbf{x}_i^{(+)}$ là một vector cột gồm hai tọa độ:

$$\mathbf{x}_i^{(+)} = \begin{bmatrix} x_{i1}^{(+)} \\ x_{i2}^{(+)} \end{bmatrix} \in \mathbb{R}^{2\times 1}.$$

Do đó, trước khi chuyển vị, dữ liệu của mỗi lớp có dạng

$$C_+ = \begin{bmatrix} (\mathbf{x}_1^{(+)})^T \\ (\mathbf{x}_2^{(+)})^T \\ \vdots \\ (\mathbf{x}_5^{(+)})^T \end{bmatrix} \in \mathbb{R}^{5\times 2}, \qquad C_- = \begin{bmatrix} (\mathbf{x}_1^{(-)})^T \\ (\mathbf{x}_2^{(-)})^T \\ \vdots \\ (\mathbf{x}_5^{(-)})^T \end{bmatrix} \in \mathbb{R}^{5\times 2}.$$

3. **Chuyển vị để mỗi cột là một điểm.**

   $\mathsf{X0} = \mathsf{class0.T}$ và $\mathsf{X1} = \mathsf{class1.T}$ tạo

   $$X_+ = C_+^T = \begin{bmatrix} | & | & & | \\ \mathbf{x}_1^{(+)} & \mathbf{x}_2^{(+)} & \cdots & \mathbf{x}_5^{(+)} \\ | & | & & | \end{bmatrix} \in \mathbb{R}^{2\times 5}, \qquad X_- = C_-^T \in \mathbb{R}^{2\times 5}.$$

   Vì vậy, điểm thứ i được truy cập bằng cột thứ i của ma trận tương ứng.

4. **Ghép hai lớp thành một tập dữ liệu.** Ghép theo chiều cột:

   $$X_{\text{points}} = [X_+ \; X_-] = \begin{bmatrix} \mathbf{x}_1^{(+)} & \cdots & \mathbf{x}_5^{(+)} & \mathbf{x}_1^{(-)} & \cdots & \mathbf{x}_5^{(-)} \end{bmatrix} \in \mathbb{R}^{2\times 10}.$$

   Tổng quát, với N điểm mỗi lớp thì $X_{\text{points}} \in \mathbb{R}^{2\times 2N}$.

5. **Tạo vector nhãn.** Năm cột đầu thuộc lớp $+1$, năm cột sau thuộc lớp -1:

   $$\mathbf{y}_+ = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \end{bmatrix}, \qquad \mathbf{y}_- = \begin{bmatrix} -1 & -1 & -1 & -1 & -1 \end{bmatrix},$$

   $$\mathbf{y} = [\mathbf{y}_+ \; \mathbf{y}_-] = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & -1 & -1 & -1 & -1 & -1 \end{bmatrix} \in \mathbb{R}^{1\times 10}.$$

   Với $2N$ điểm, vector nhãn tổng quát có dạng

   $$\mathbf{y} = \begin{bmatrix} \underbrace{1,\ldots,1}_{N \text{ lần}}, \underbrace{-1,\ldots,-1}_{N \text{ lần}} \end{bmatrix} \in \mathbb{R}^{1\times 2N}.$$

6. **Bổ sung hệ số tự do.** Để biểu diễn đường phân cách $w_0 + w_1 x_1 + w_2 x_2 = 0$ dưới dạng tích vô hướng, thêm một hàng toàn số 1:

   $$\mathbf{1}_{1\times 10} = \begin{bmatrix} 1 & 1 & \cdots & 1 \end{bmatrix}, \qquad X = \begin{bmatrix} \mathbf{1}_{1\times 10} \\ X_{\text{points}} \end{bmatrix} \in \mathbb{R}^{3\times 10}.$$

   Cột thứ j của X là vector mở rộng

   $$\bar{\mathbf{x}}_j = \begin{bmatrix} 1 \\ x_{j1} \\ x_{j2} \end{bmatrix} \in \mathbb{R}^3, \qquad j = 1,\ldots,10.$$

   Do đó, sau khối 1, dữ liệu được đưa vào PLA dưới dạng cặp $(\bar{\mathbf{x}}_j, y_j)$, với $X \in \mathbb{R}^{3\times 10}$ và $\mathbf{y} \in \mathbb{R}^{1\times 10}$. Tổng quát với N điểm mỗi lớp, $X \in \mathbb{R}^{(d+1)\times 2N}$; trong bài này $d = 2$, nên $d+1 = 3$.

2.2 Block 2: Các hàm của PLA

Khối này gồm năm hàm. Chúng tách riêng phép lấy dấu, dự đoán một điểm, dự đoán cả tập, kiểm tra hội tụ và vòng lặp cập nhật.

2.2.1 Hàm dấu

Listing 2: Hàm dấu dùng trong luật dự đoán.

```
48 def sign_of(value):
49     """Giong np.sign: > 0 -> 1, < 0 -> -1, = 0 -> 0."""
50     if value > 0:
51         return 1
52     if value < 0:
53         return -1
54     return 0
```

2.2.2 Dự đoán một điểm

Listing 3: Tính sign($\mathbf{w}^\top \mathbf{x}$) cho một cột của X.

```
57 def predict_one(w, x):
58     """
59     Du doan nhan cua 1 diem.
60     w: vector trong so, shape (3, 1)
61     x: 1 diem [1, x, y], shape (3, 1)
62 
63     Tinh tich vo huong: w0*1 + w1*x + w2*y
64     roi lay dau.
65     """
66     s = 0.0
67     for k in range(len(x)):
68         s = s + w[k, 0] * x[k, 0]
69     return sign_of(s)
```

w và x đều có shape $(3, 1)$. Vòng lặp cộng $w_0 \cdot 1 + w_1 x_1 + w_2 x_2$, tức tích vô hướng $\mathbf{w}^\top \bar{\mathbf{x}}$, rồi trả về dấu của tổng.

2.2.3 Dự đoán toàn dataset

Listing 4: Áp dụng dự đoán cho mọi cột của ma trận dữ liệu.

```
72 def predict_all(w, X_data):
73     """Du doan nhan tat ca cac diem."""
74     n_points = X_data.shape[1]
75     preds = []
76     for i in range(n_points):
77         xi = X_data[:, i].reshape(3, 1)
78         preds.append(predict_one(w, xi))
79     return preds
```

X_data.shape[1] là số điểm. Cột i được reshape(3, 1) rồi đưa vào predict_one. Kết quả là một danh sách Python dài bằng số điểm, không phải mảng NumPy; các hàm phía sau chỉ cần so sánh từng phần tử với nhãn.

2.2.4 Kiểm tra hội tụ

Listing 5: Dừng khi không còn điểm nào bị gán sai nhãn.

```python
def has_converged(X_data, y_data, w):
    """True neu moi diem deu duoc phan loai dung."""
    preds = predict_all(w, X_data)
    n_points = X_data.shape[1]
    for i in range(n_points):
        if preds[i] != y_data[0, i]:
            return False
    return True
```

Hàm trả về True chỉ khi mọi dự đoán trùng nhãn. So sánh là $\neq$, nên dự đoán 0 không khớp $+1$ hay -1 và bị tính là chưa hội tụ. Đây đúng định nghĩa tách được tuyến tính ở chương trước: tồn tại $\mathbf{w}$ sao cho dấu của $\mathbf{w}^\top \bar{\mathbf{x}}_i$ bằng y_i với mọi i.

2.2.5 Vòng lặp cập nhật

Listing 6: PLA xáo trộn thứ tự điểm và sửa trọng số mỗi lần dự đoán sai.

```python
def perceptron(X_data, y_data, w_init):
    """
    Perceptron Learning Algorithm (PLA).

    Moi vong:
    - xao tron thu tu cac diem
    - neu diem bi sai: w_new = w_old + y * x
    Lap den khi khong con diem sai.
    """
    w_history = [w_init]
    mis_points = []
    n_points = X_data.shape[1]

    while True:
        # xao tron thu tu diem
        order = np.random.permutation(n_points)

        for i in range(n_points):
            idx = order[i]
            xi = X_data[:, idx].reshape(3, 1)
            yi = y_data[0, idx]

            pred = predict_one(w_history[-1], xi)
            if pred != yi:
                # diem nay dang bi phan loai sai -> cap nhat w
                mis_points.append(idx)
                w_new = w_history[-1] + yi * xi
                w_history.append(w_new)

        if has_converged(X_data, y_data, w_history[-1]):
            break

    return w_history, mis_points
```

w_history bắt đầu bằng trọng số khởi tạo. mis_points ghi chỉ số của những điểm gây ra một lần cập nhật, theo đúng thứ tự các lần đó xảy ra.

Mỗi vòng while xáo một hoán vị của $0, 1, \ldots, 9$. Với từng chỉ số trong hoán vị, chương trình lấy cột tương ứng của X và nhãn ở hàng 0 của y. Nếu dấu dự đoán khác nhãn, điểm đó được ghi vào mis_points và trọng số được sửa bằng

$$\mathbf{w}_{\text{mới}} = \mathbf{w}_{\text{cũ}} + y_i \bar{\mathbf{x}}_i,$$

đúng luật (2). Vector mới được nối vào cuối w_history.

Kiểm tra hội tụ nằm *sau* cả lượt duyệt, không nằm ngay sau từng điểm. Vì vậy một lượt có thể sửa trọng số nhiều lần. Khi has_converged đúng, hàm trả về toàn bộ lịch sử trọng số và danh sách điểm đã gây cập nhật. Nếu dữ liệu không tách được tuyến tính, while True không thoát.

2.3 Block 3: Chạy PLA

Listing 7: Khởi tạo trọng số ngẫu nhiên và chạy PLA trên tập vừa sinh.

```
# 3. Chay PLA
# ============================================================
d = X.shape[0]  # = 3
w_init = np.random.randn(d, 1)
w_history, mis_points = perceptron(X, y, w_init)
print(mis_points)
```

d $= 3$ là số hàng của X. Trọng số ban đầu là một cột Gauss chuẩn, shape $(3, 1)$. Vì seed đã đặt ở khối 1, vector này không đổi giữa các lần chạy.

Với seed 2, chương trình in ra 20 chỉ số điểm bị phân loại sai trước khi hội tụ:

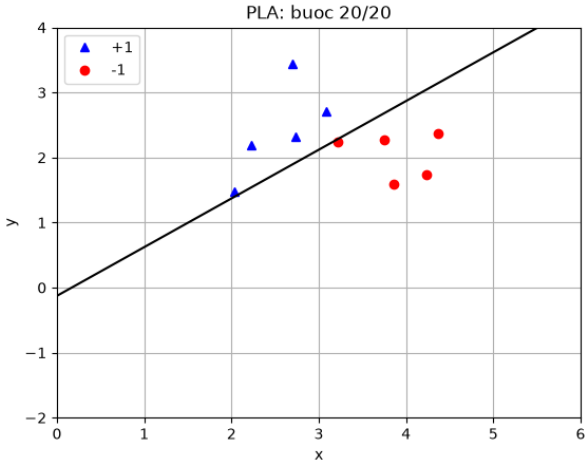
```
$ python basic.py
[np.int32(3), np.int32(8), np.int32(0), np.int32(9), np.int32(4), np.int32(3), np.int32(6), np.int32(2), np.int32(7), np.int32(1), np.int32(6), np.int32(4),
 np.int32(8), np.int32(0), np.int32(5), np.int32(1), np.int32(5), np.int32(1), np.int32(8), np.int32(4)]
```

Hình 3: *Kết quả 20 điểm bị phân loại là sai trước khi hội tụ.*

Chỉ số 0 đến 4 thuộc lớp $+1$, chỉ số 5 đến 9 thuộc lớp -1. Cả hai lớp đều xuất hiện trong danh sách, nghĩa là biên ban đầu cắt qua cả hai đám điểm và phải được kéo dần. w_history có 21 phần tử: một vector khởi tạo cộng 20 lần cập nhật.

2.4 Block 4: Vẽ đường phân tách

2.5 Block 5: Hoạt họa từng lần cập nhật



Hình 4: *Kết quả chạy PLA với $N = 5$ sau 20 lần cập nhật*

Nhìn lại cả năm khối, dữ liệu ở khối 1 tạo đúng cặp $(\bar{\mathbf{x}}, y)$ của mô hình tuyến tính; khối 2 thực hiện (1) và (2) cho đến khi dấu dự đoán trùng nhãn; khối 3 chạy một lần với trọng số ngẫu nhiên; khối 4 và khối 5 chỉ đổi **w** thành đường thẳng và chiếu lại lịch sử các lần sai.